

# DMV CoreTech Intern Task 1

## Python Scientific Calculator

**Name:** Arya Gupta

**Domain:** Data Science

**GitHub Repository Link:** <https://github.com/iaryaguptaa/Python-Scientific-Calculator>

### Project Implementation

- **Modular Architecture:** Instead of a single script, I separated each mathematical operation (addition, division, logarithms, etc.) into its own independent module. This mimics the architecture of real-world Data Science libraries like Pandas or NumPy.
- **Centralized Interface:** The main.py file acts solely as the user interface, importing the required modules and handling the user's menu choices.
- **Robust Error Handling:** I implemented try...except ValueError blocks to catch invalid inputs (like typing letters instead of numbers) and added mathematical edge-case handling (e.g., preventing crashes when dividing by zero or taking the log of a negative number).

## Output Screenshots

```
Select an operation (1-11): 4
Enter first number: 15
Enter second number: 0
Result: Error: Cannot divide by zero!
```

```
Scientific Calculator
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Power (x^y)
6. Square Root (√x)
7. Trigonometry (sin, cos, tan)
8. Logarithms (ln, lg)
9. Angle Conversions (Deg<->Rad)
10. Pi (π)
11. Exit
```

```
Select an operation (1-11): █
```

```
Select an operation (1-11): 8
Enter the number: 0
Choose (a) ln, (b) lg: b
Result: Error: Logarithm is undefined for zero or negative numbers.
```

```
Scientific Calculator
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Power (x^y)
6. Square Root (√x)
7. Trigonometry (sin, cos, tan)
8. Logarithms (ln, lg)
9. Angle Conversions (Deg<->Rad)
10. Pi (π)
11. Exit
```

```
Select an operation (1-11): █
```

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

```
~/Users/ARYA/Desktop/DS Internship projects/1) calculator 26 May 2026/main.py"
```

```
Scientific Calculator
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Power (x^y)
6. Square Root (√x)
7. Trigonometry (sin, cos, tan)
8. Logarithms (ln, lg)
9. Angle Conversions (Deg<->Rad)
10. Pi (π)
11. Exit
```

```
Select an operation (1-11): 1
Enter first number: 5.5
Enter second number: -2.2
Result: 3.3
```

```
Scientific Calculator
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Power (x^y)
6. Square Root (√x)
7. Trigonometry (sin, cos, tan)
8. Logarithms (ln, lg)
9. Angle Conversions (Deg<->Rad)
10. Pi (π)
11. Exit
```

```
Select an operation (1-11): █
```

## 1. Basic to Intermediate Level Questions

### Q1. What is the difference between a List and a Tuple in Python?

The primary difference is mutability.

- **Lists** are mutable, meaning their elements can be changed, added, or removed after creation. They are defined using square brackets `[]`.
- **Tuples** are immutable, meaning once they are created, their elements cannot be modified. They are defined using parentheses `()`. Because they are immutable, tuples are faster and more memory-efficient than lists.

### Q2. Explain the difference between mutable and immutable data types. Give examples.

- **Mutable** data types allow their state or contents to be changed in-place without creating a new object in memory. *Examples:* Lists, Dictionaries, Sets.
- **Immutable** data types do not allow any changes after they are created. Any operation that attempts to modify them will instead create a brand new object in memory. *Examples:* Strings, Integers, Floats, Tuples.

### Q3. What is a Dictionary in Python and how is it structured?

A dictionary is an unordered, mutable collection of data values stored as **key-value pairs**. It is defined using curly braces `{}`. Each key in a dictionary must be unique and of an immutable data type (like a string or integer), while the values can be of any data type and can be duplicated. They are heavily used in Data Science for JSON processing and data mapping.

## 2. Concept-Based Questions

### Q4. What is the difference between 'break', 'continue', and 'pass'?

- **break** : Terminates the loop entirely and transfers execution to the statement immediately following the loop.
- **continue** : Skips the remaining code inside the loop for the current iteration and forces the loop to move to the next iteration.
- **pass** : A null operation. It does nothing when executed. It is used as a placeholder in empty functions or loops to prevent syntax errors.

### Q5. How does Python manage memory?

Python manages memory automatically using two primary mechanisms:

1. **Reference Counting**: Python keeps track of the number of references to every object in memory. If an object's reference count drops to zero, the memory is immediately freed.
2. **Garbage Collection**: A background process that specifically looks for and cleans up "reference cycles" (e.g., when object A references object B, and object B references object A, preventing their reference counts from ever reaching zero).

### Q6. What is the purpose of the `__init__` method in object-oriented programming?

The `__init__` method is a special method (also known as a constructor) in Python classes. It is automatically called when a new instance (object) of a class is created. Its primary purpose is to initialize the attributes of the object, assigning initial values to the object's properties.

### 3. Output-Based Questions

**Q7. What will be the output of the following code? Explain why.**

```
list_a = [1, 2, 3]
list_b = list_a
list_b.append(4)
print("list_a:", list_a)
print("list_b:", list_b)
```

**Output:**

```
list_a: [1, 2, 3, 4]
```

```
list_b: [1, 2, 3, 4]
```

**Explanation:** In Python, assigning an existing list to a new variable ( `list_b = list_a` ) does not create a copy of the list. Instead, both variables point to the exact same object in memory. Therefore, modifying `list_b` also modifies `list_a` .

**Q8. What will be the output of the following string slicing operation?**

```
word = "DataScience"
print(word[::-1])
```

**Output:**

```
ecneicSataD
```

**Explanation:** The slicing syntax is `[start:stop:step]` . Leaving start and stop blank tells Python to cover the entire string, and a step of `-1` tells it to traverse the string backwards, effectively reversing it.

### Q9. Analyze the dictionary output.

```
my_dict = {"a": 1, "b": 2, "a": 3}
print(my_dict)
```

**Output:**

```
{'a': 3, 'b': 2}
```

**Explanation:** Dictionary keys must be unique. If a key is repeated during dictionary creation, Python will overwrite the previous value with the last value assigned to that key.

## 4. Short Coding Questions

### Q10. Write a list comprehension to create a list of the squares of all even numbers from 1 to 10.

```
squares_of_evens = [x**2 for x in range(1, 11) if x % 2 == 0]
print(squares_of_evens)
# Output: [4, 16, 36, 64, 100]
```

### Q11. Write a short function using Exception Handling to safely divide two numbers.

```
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        return "Error: Cannot divide by zero!"
    except TypeError:
        return "Error: Both inputs must be numbers."

print(safe_divide(10, 0))
```

**Q12. Write a short Python program to check if a string is a palindrome.**

```
def is_palindrome(text):  
    # Convert to lowercase to ensure case-insensitivity  
    text = text.lower()  
    return text == text[::-1]  
  
print(is_palindrome("Radar")) # Output: True
```

## 5. Frequently Asked Interview Questions

**Q13. What is the difference between the '==' operator and the 'is' keyword?**

- **==** (Equality Operator): Checks if the **values** of two operands are equal.
- **is** (Identity Operator): Checks if two operands refer to the exact same **object in memory**.

*Example:* Two identical lists will return True for **==**, but False for **is** because they exist at different memory addresses.

**Q14. What are \*args and \*\*kwargs in Python functions?**

They are used to pass a variable number of arguments to a function.

- **\*args** (Non-Keyword Arguments): Allows passing multiple positional arguments. Inside the function, they are gathered into a **tuple**.
- **\*\*kwargs** (Keyword Arguments): Allows passing multiple named (keyword) arguments. Inside the function, they are gathered into a **dictionary**.

**Q15. Explain the concept of 'self' in Python object-oriented programming.**

**self** represents the instance of the class itself. When defining methods inside a class, **self** must be the first parameter. It binds the attributes and methods to the specific object being created, allowing the object to access its own data. It is similar to **this** in Java or C++.

### Q16. What is the safest way to open and read a file in Python?

The safest and most Pythonic way to handle files is using the `with` statement (a context manager). It ensures that the file is automatically closed as soon as the block of code exits, even if an exception occurs during file processing.

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File is automatically closed here
```